

Avance Semana 15

Equipo EPS - 21/05/2026



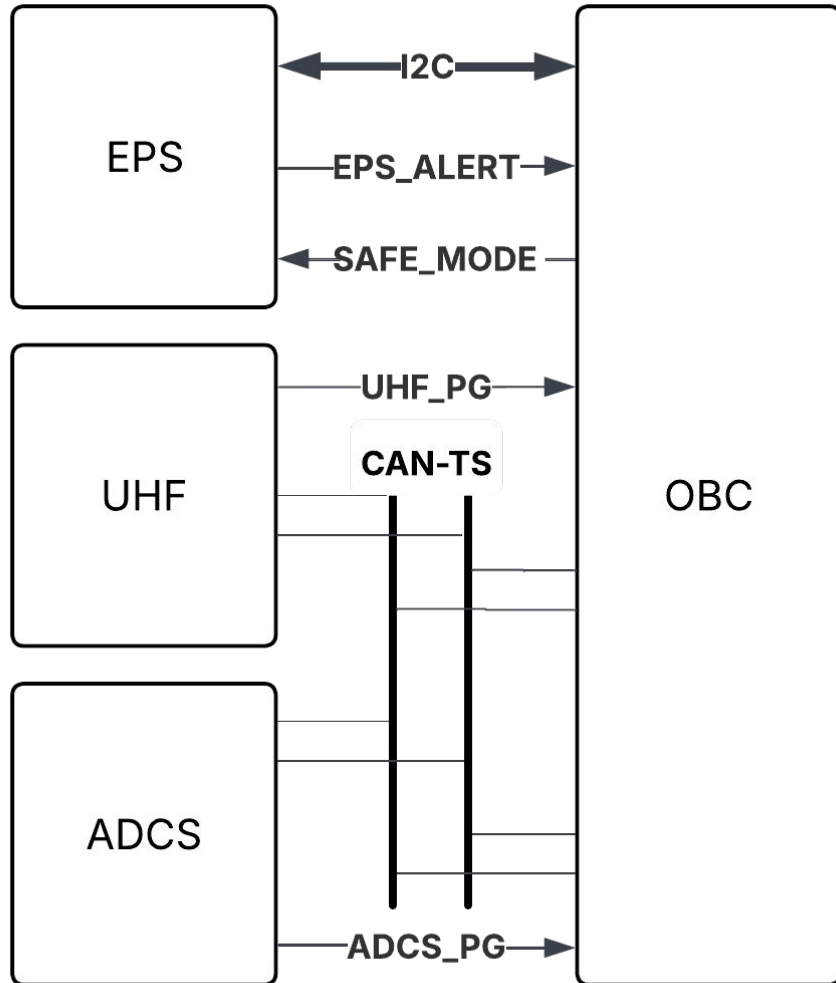
PUCP



Universidad Católica
San Pablo

Tareas para esta semana

- Verificar sensores mediante mapa de memoria por I2C (100%)
- Verificar sensores mediante mapa de memoria por UART (50%)
- Programar protocolo y comandos de comunicación EPS-OBC propio por I2C en el MSP430 (100%).
- Programar librerías del protocolo para para el STM32 (75%)
- Verificar heaters y RTDs (0%).
- Elaborar un plan de pruebas para baterías y verificar fuel gauge (0%).



- Idealmente todos los GPIO deberían ser conectados con resistencias pull-up.
 - Se evita estados indeterminados al prender MCU: Activo en baja.
 - Mayor inmunidad ante ruido EMI.
- En nuestro caso `SAFE_MODE`:
 - Es entrada con pull-up en MSP430.
 - Esa salida open-drain en STM32.

Protocolo OBC-EPS INTISAT

- **Modificaciones de anterior versión:**

- Se quitaron comandos SET_POWER_ON y SET_POWER_OFF.
- Se pide actualización completa de power_en_reg. Menos errores y actualización en caso de SEU.
- Dummy byte

OpCode (CMD)	Command Name	Description	OBC TX Frame (Write Phase)	EPS TX Frame (Read Phase)
0x10	REQ_FULL_TLM	Requests the complete telemetry structure (IDs 0 to 49).	[0x10] + [CRC]	[100 bytes of complete data] + [CRC] (Or [0xFE] if Busy)
0x11	REQ_BEACON_TLM	Requests basic telemetry data for the Beacon.	[0x11] + [CRC]	[15 bytes of beacon data] + [CRC] (Or [0xFE] if Busy)
0x12	REQ_SINGLE_PARAM	Reads a specific ID from the data structure.	[0x12] + [ID] + [CRC]	[Echo ID] + [N Data Bytes] + [CRC] (Or [0xFE] if Busy)
0x20	WR_SINGLE_PARAM	Writes a value to a specific ID	[0x20] + [ID] + [N Data Bytes] + [CRC]	[ACK status (1 byte)] + [CRC] OK: ACK status = 0x00 Invalid ID/Error: ACK status = 0xFF (Or [0xFE] if Busy)
0x40	READ_ANOMALY	Reads the emergency error code (requested after EPS_ALERT interrupt).	[0x40] + [CRC]	[Error code (2 bytes)] + [CRC]

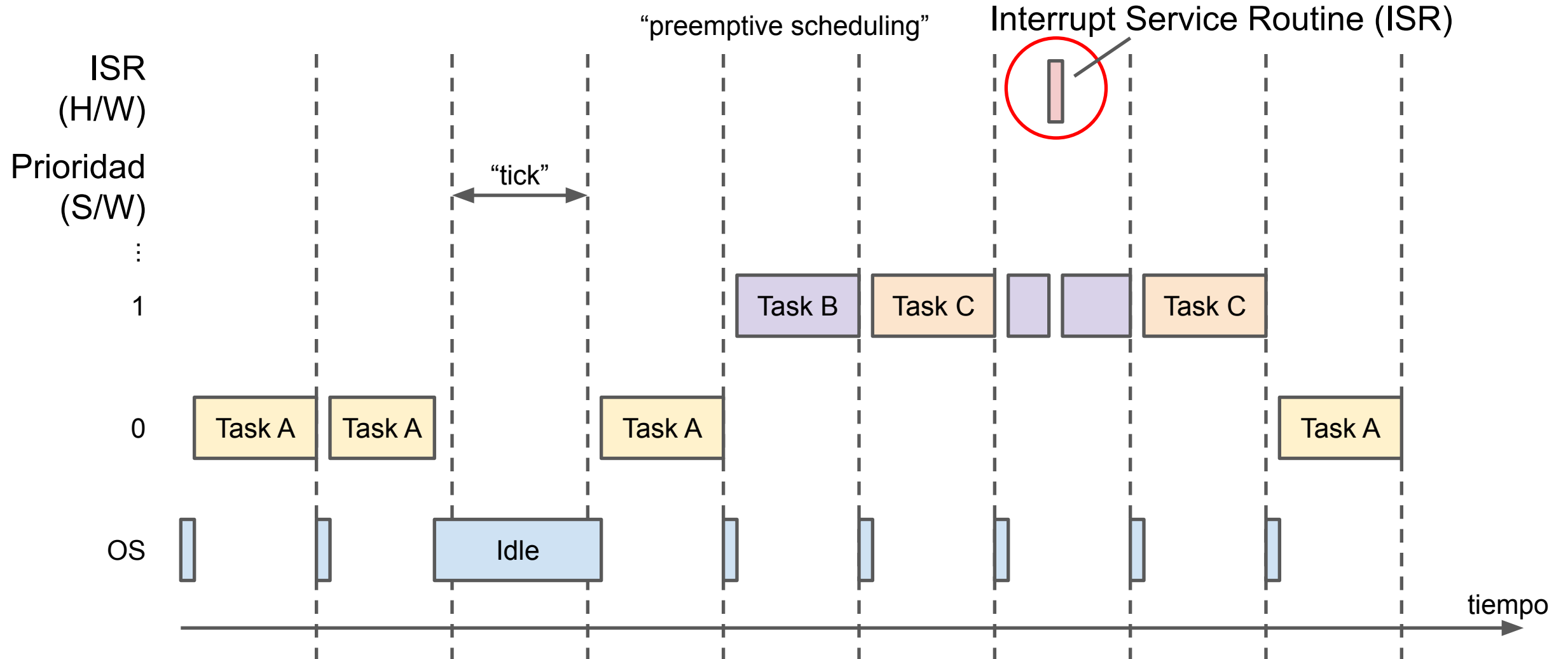
Protocolo OBC-EPS INTISAT

- Se agregaron 2 nuevos elementos a la estructura de datos

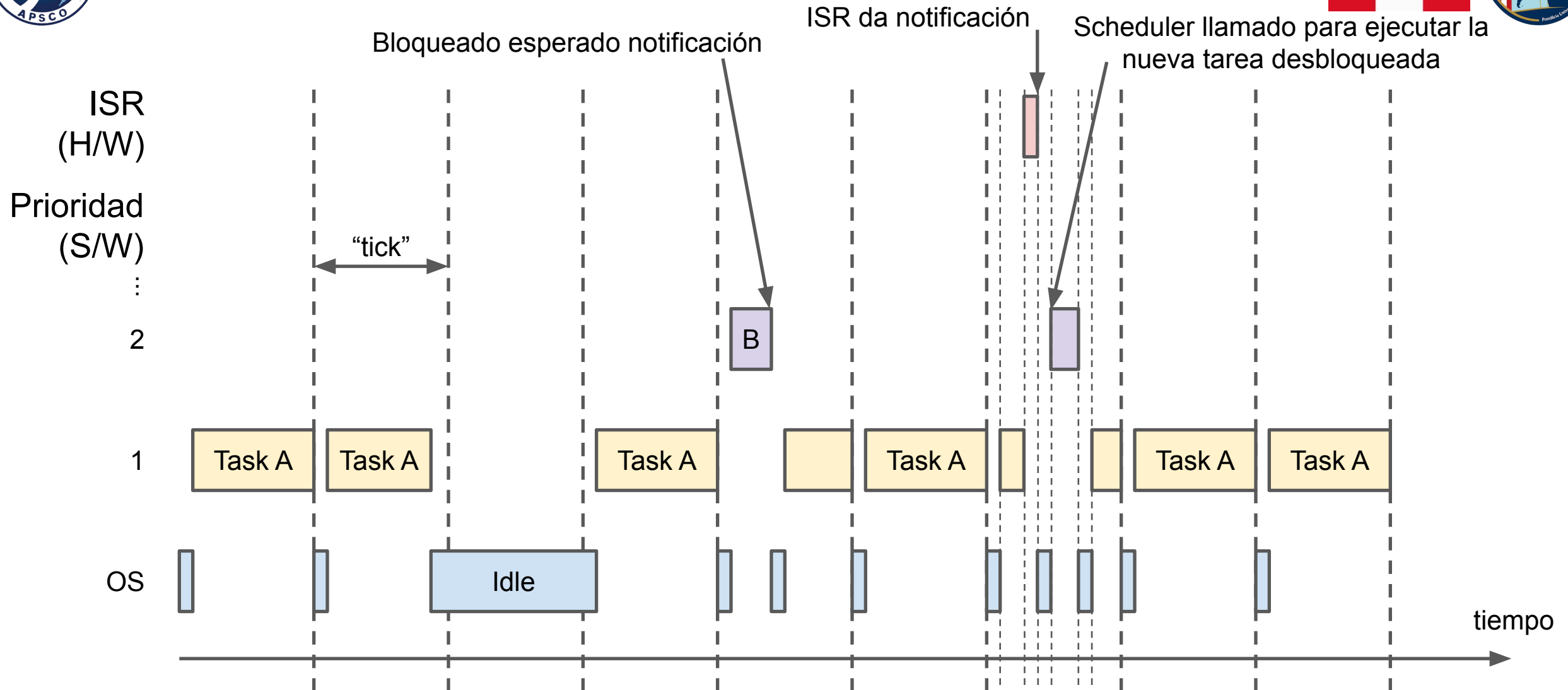
Power switches enable register		
Type	uint8_t	
ID	47	
Variable name	<u>power_en_reg</u>	
Access	Read/write	
Word format		
Bit	Field name	Description
7 (MSB)	reserved	Reserved for future use. Write as 0, ignore on read.
6	en_3v3_psw0	Active-high enable for power switch 0 derived from the 3v3_2a regulator
5	en_3v3_psw1	Active-high enable for power switch 1 derived from the 3v3_2a regulator
4	en_5v_3a	Active-high enable for the 5v_3a buck regulator
3	en_5v_psw0	Active-high enable for power switch 0 derived from the 5v_5a_1 regulator
2	en_5v_psw1	Active-high enable for power switch 1 derived from the 5v_5a_1 regulator
1	en_5v_psw2	Active-high enable for power switch 2 derived from the 5v_5a_0 regulator
0 (LSB)	en_5v_psw3	Active-high enable for power switch 3 derived from the 5v_5a_0 regulator

Protocolo OBC-EPS INTISAT

Buck regulators enable register		
Type	uint8_t	
ID	48	
Variable name	<u>buck_en_reg</u>	
Description	Active-high status of the buck regulators. These regulators do not route directly to the PC/104 header; instead, they supply the power switches. To maximize power saving, each buck regulator is automatically disabled internally if all of its derived power switches are turned off.	
Access	Read	
Word format		
Bit	Field name	Description
7:3	Reserved	Reserved for future use. Reads as 0.
2	en_3v3_2a	Active-high enable for the 3v3_2a buck regulator
1	en_5v_5a_1	Active-high enable for the 5v_5a_1 buck regulator
0 (LSB)	en_5v_5a_0	Active-high enable for the 5v_5a_0 buck regulator



Deferred Interrupt Processing



Deferred Interrupt Processing

```
void vTaskParamServer(void *pvParameters){
    BaseType_t result;
    uint32_t notified_value;
    xEventGroupWaitBits(task_startup_status, TASK_STARTUP_DONE, pdFALSE, pdTRUE, pdMS_TO_TICKS(TASK_PARAM_SERVER_INIT_TIMEOUT_MS));
    while(1)
    {
        result = xTaskNotifyWait(0UL, 0xFFFFFFFFUL, &notified_value, pdMS_TO_TICKS(TASK_PARAM_SERVER_MAX_BLOCK_TIME_MS));

        uint8_t adr = 0;
        uint32_t val = 0;
        uint8_t cmd = 0;

        if (result == pdPASS)
        {
            /* Process interrupt from SAFE_MODE ISR*/
            if ((notified_value & NOTIFICATION_VALUE_FROM_SAFE_MODE_ISR) != 0){
            }
            /* Process interrupt from I2C ISR. */
            if ((notified_value & NOTIFICATION_VALUE_FROM_I2C_ISR) != 0){
            }
            /* Process interrupt from UART ISR. */
            if ((notified_value & NOTIFICATION_VALUE_FROM_UART_ISR) != 0){
            }
        }
    }
}
```

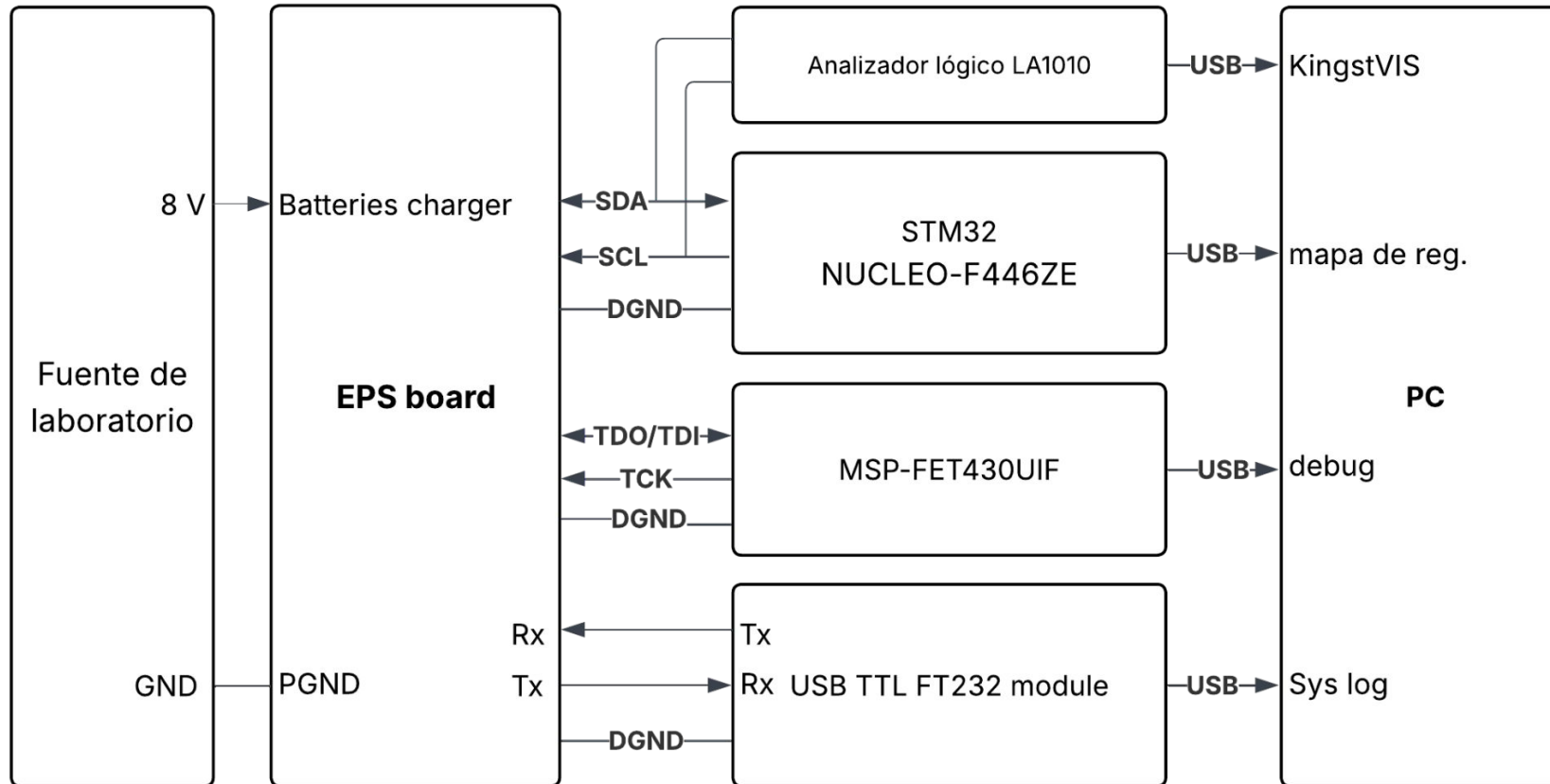
Deferred Interrupt Processing

```
void gpio_port2_pin6_isr(void){
    safe_mode();          /**< Turn off all power switches and regulators. */
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    xTaskNotifyFromISR(xTaskParamServerHandle, NOTIFICATION_VALUE_FROM_SAFE_MODE_ISR, eSetBits, &xHigherPriorityTaskWoken);
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}

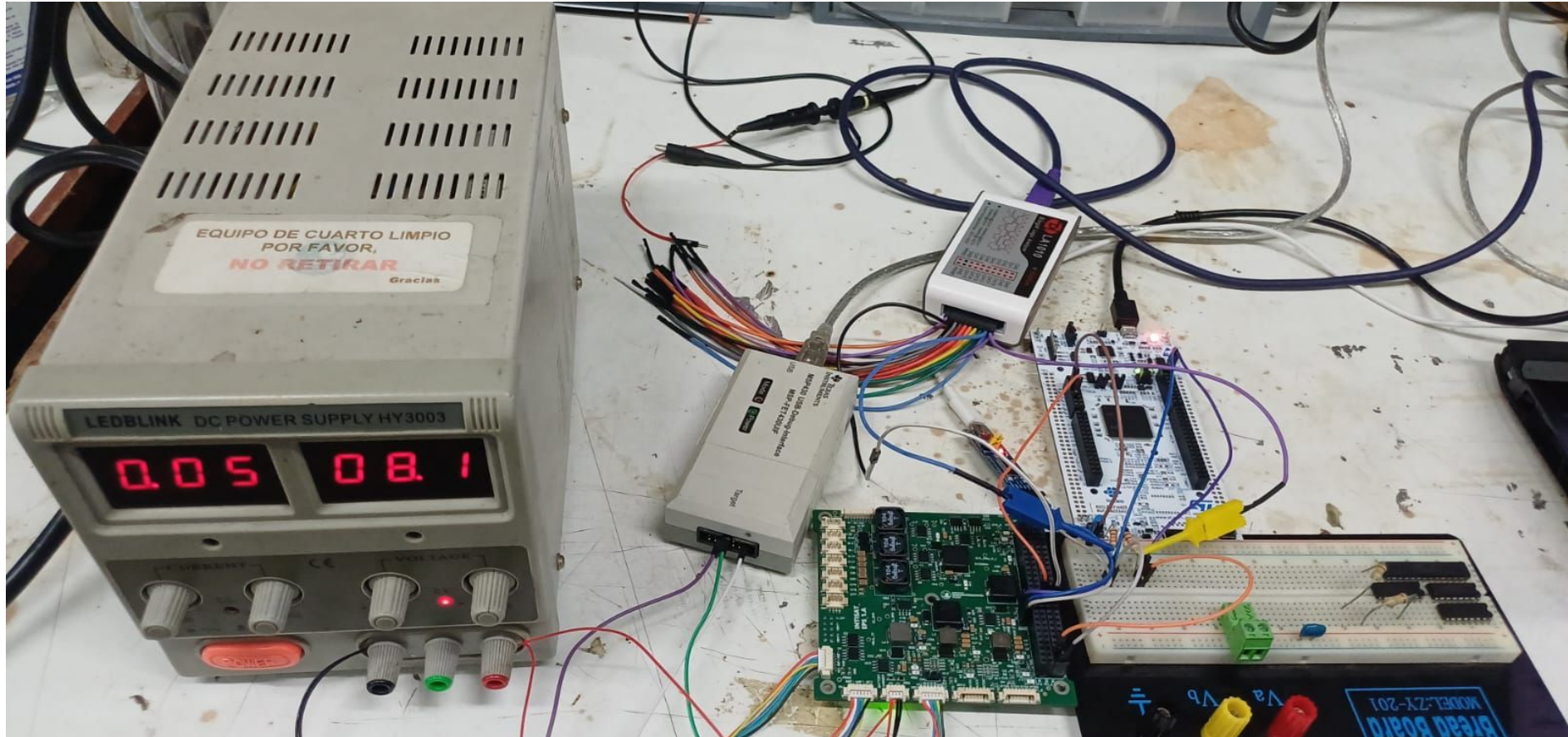
void i2c_slave_notify_from_i2c_rx_isr(void){
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    xTaskNotifyFromISR(xTaskParamServerHandle, NOTIFICATION_VALUE_FROM_I2C_ISR, eSetBits, &xHigherPriorityTaskWoken);
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}

void uart_interrupt_notify_from_rcv_isr(void){
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    xTaskNotifyFromISR(xTaskParamServerHandle, NOTIFICATION_VALUE_FROM_UART_ISR, eSetBits, &xHigherPriorityTaskWoken);
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}
```

Probar protocolo EPS-OBC propio por I2C



Probar protocolo EPS-OBC propio por I2C



Programar protocolo EPS-OBC propio por I2C

LA1010 Connected - KingstVIS



```

Setup Write to [0x36] + ACK
0x00 + ACK
0x00 + ACK
Setup Read to [0x36] + ACK
0x00 + ACK
0x00 + ACK
0x00 + ACK
0x00 + ACK
0x09 + NAK
Setup Write to [0x36] + ACK
0x01 + ACK
0x07 + ACK
Setup Read to [0x36] + ACK
0x0E + ACK
0x01 + ACK
0x00 + NAK
Setup Write to [0x36] + ACK
0x02 + ACK
0x0E + ACK
Setup Read to [0x36] + ACK
0x00 + ACK
0x02 + ACK
0x00 + NAK
Setup Write to [0x36] + ACK
0x03 + ACK
0x09 + ACK
    
```

PuTTY (inactive)

```

ID: 0: 3992977412
ID: 1: 65025
ID: 2: 2
ID: 3: 0
ID: 4: 4
ID: 5: 5
ID: 6: 262
ID: 7: 263
ID: 8: 264
ID: 9: 9
ID: 10: 10
ID: 11: 11
ID: 12: 12
ID: 13: 13
ID: 14: 0
ID: 15: 0
ID: 16: 0
ID: 17: 17
ID: 18: 5906
ID: 19: 387163557
ID: 20: 2652153253
ID: 21: 3071649189
ID: 22: 3575031205
ID: 23: 1930827685
    
```

Tareas para la siguiente semana

- Probar todos los reguladores DC-DC con resistencias de cemento o alta potencia.
- Evaluar estabilidad de osciladores externos a 32 MHz.
- Probar el protocolo de comunicación propio por UART e I2C
- Investigar sobre funcionamiento de Bootstrap Loader y una rutina de reprogramación. Permite reprogramación por UART/I2C.



PUCP



Universidad Católica
San Pablo